Detailed Report: Technical Analysis Toolkit for Weather Data

Introduction

This project is an Object-Oriented Programming (OOP) midterm assignment focused on developing a technical analysis toolkit for weather data visualization and prediction. The goal is to process historical weather data, specifically temperature data collected at an hourly rate across European countries from 1980 to 2019, and generate meaningful insights through candlestick charting and data analysis techniques. The project is designed as a command-line application, allowing users to compute candlestick data, visualize it in text-based plots, apply data filters, and predict future temperature trends based on historical data.

The project involves several key components, each implemented using C++ with a strong focus on OOP principles. These components include classes for data parsing, candlestick calculation, filtering, plotting, and prediction. The program handles various tasks such as reading and processing large dataset, calculating statistical values, and generating meaningful visual representations of the data.

---

2. Project Overview

The project is divided into four primary tasks:

1. **Task 1: Compute Candlestick Data**
   This involves converting raw temperature data into candlestick summaries that represent daily, monthly, or yearly weather trends. The candlestick format includes:
   - **Open**: The first recorded temperature of the period.
   - **High**: The highest temperature recorded during the period.
   - **Low**: The lowest temperature recorded during the period.
   - **Close**: The last recorded temperature of the period.
2. **Task 2: Create a Text-Based Plot**
   The project includes a module that plots candlestick data in a text-based format for visualization. The chart dynamically scales based on temperature ranges and allows the user to view trends over time.
3. **Task 3: Filter Data**
   Users can filter candlestick data based on criteria such as date range, temperature range, and specific months.
4. **Task 4: Predict Future Temperatures**
   The prediction module uses a moving average technique to forecast future temperatures based on historical data.

Task 1: Compute Candlestick Data

Task 1 focuses on computing candlestick data from hourly temperature readings for a specific time frame (daily, monthly, or yearly). This involves using several classes, constructors, functions, and objects to perform the calculations and store the results. Below is a detailed explanation of each component involved in Task 1.

Classes Used in Task 1

1. **Candlestick Class**
   - Represents a single candlestick object with fields for date, open, high, low, and close values.
   - This class provides the data structure to store candlestick data.

   Fields:

   - `std::string date`: The date associated with the candlestick.
   - `double open`: The opening temperature value for the time frame.
   - `double high`: The highest temperature value for the time frame.
   - `double low`: The lowest temperature value for the time frame.
   - `double close`: The closing temperature value for the time frame.

Constructors:

- **Default Constructor:** Initializes the `open`, `high`, `low`, and `close` values to zero.

```
// Default constructor initializing values to zero
Candlestick() : open(0), high(0), low(0), close(0) {}
```

- **Parameterized Constructor:** Allows initializing the `Candlestick` object with specific values.

```
// Parameterized constructor to initialize candlestick data with specific values
Candlestick(const std::string &date, double open, double high, double low, double close)
    : date(date), open(open), high(high), low(low), close(close) {}
```

Methods:

- `void displayRow() const`: Displays the candlestick data in a formatted row for debugging and visualization.

```
// Method to display candlestick data in a formatted row with specific column widths
void Candlestick::displayRow() const
{
    // Output the date and candlestick values (open, close, high, low) with proper spacing using setw
    std::cout << std::setw(12) << date
              << std::setw(10) << open
              << std::setw(10) << close
              << std::setw(10) << high
              << std::setw(10) << low
              << std::endl;
}
```

2. **CandlestickCalculator Class**

- This class provides methods to compute daily, monthly, and yearly candlestick data from temperature readings.

Functions in CandlestickCalculator Class:

- **`calculateDailyCandlesticks`** Function:**

    - Takes a vector of hourly temperature readings as input.
    - Processes the data in 24-hour chunks to compute daily candlestick values.
    - Uses `std::max_element`, `std::min_element`, and `std::accumulate` to calculate the high, low, and close values.

    Logic for Calculating Daily Candlesticks:

    - **Open:** The previous day's close value.
    - **High:** The maximum temperature in the 24-hour period.
    - **Low:** The minimum temperature in the 24-hour period.
    - **Close:** The average temperature for the 24-hour period.

```
// Iterate through the hourly data in 24-hour chunks
for (size_t i = 0; i < hourlyData.size(); i += 24)
{
    Candlestick candle;

    // The "Open" value is the previous day's average temperature
    candle.open = previousClose;

    // The "High" value is the maximum temperature of the day
    candle.high = *std::max_element(hourlyData.begin() + i, hourlyData.begin() + std::min(i + 24, hourlyData.size()));

    // The "Low" value is the minimum temperature of the day
    candle.low = *std::min_element(hourlyData.begin() + i, hourlyData.begin() + std::min(i + 24, hourlyData.size()));

    // The "Close" value is the average temperature of the day
    candle.close = std::accumulate(hourlyData.begin() + i, hourlyData.begin() + std::min(i + 24, hourlyData.size()), 0.0) / 24;

    // Add the candlestick to the daily candlesticks vector
    dailyCandlesticks.push_back(candle);

    // Update the previous close value for the next day's open
    previousClose = candle.close;
}
return dailyCandlesticks;
}
```

- **`calculateMonthlyCandlesticks`** Function:**

    - Takes a vector of daily candlesticks as input.
    - Processes the data in 30-day chunks to compute monthly candlestick values.

    Logic for Calculating Monthly Candlesticks:

    - **Open:** The previous month's close value.
    - **High:** The maximum high value across 30 days.
    - **Low:** The minimum low value across 30 days.
    - **Close:** The average of the daily close values over 30 days.

```cpp
std::vector<Candlestick> monthlyCandlesticks;
double previousClose = 0.0;  // Initialize the previous month's closing value to 0.0

// Iterate through the daily candlesticks in 30-day chunks
for (size_t i = 0; i < dailyCandlesticks.size(); i += 30)
{
    Candlestick candle;

    // The "Open" value is the previous month's average temperature
    candle.open = previousClose;

    // The "High" value is the highest temperature recorded in the month
    candle.high = std::max_element(dailyCandlesticks.begin() + i, dailyCandlesticks.begin() + std::min(i + 30, dailyCandlesticks.size()),
                                   [](const Candlestick &a, const Candlestick &b) { return a.high < b.high; })
                  ->high;

    // The "Low" value is the lowest temperature recorded in the month
    candle.low = std::min_element(dailyCandlesticks.begin() + i, dailyCandlesticks.begin() + std::min(i + 30, dailyCandlesticks.size()),
                                  [](const Candlestick &a, const Candlestick &b) { return a.low < b.low; })
                 ->low;

    // The "Close" value is the average temperature of the month
    candle.close = std::accumulate(dailyCandlesticks.begin() + i, dailyCandlesticks.begin() + std::min(i + 30, dailyCandlesticks.size()), 0.0,
                                   [](double sum, const Candlestick &c) { return sum + c.close; }) / 30;

    // Add the candlestick to the monthly candlesticks vector
    monthlyCandlesticks.push_back(candle);

    // Update the previous close value for the next month's open
    previousClose = candle.close;
}
```

- **`calculateYearlyCandlesticks`** Function:**
    - Takes a vector of monthly candlesticks as input.
    - Processes the data in 12-month chunks to compute yearly candlestick values.

Logic for Calculating Yearly Candlesticks:

- **Open:** The previous year's close value.
- **High:** The maximum high value across 12 months.
- **Low:** The minimum low value across 12 months.
- **Close:** The average of the monthly close values over 12 months.

```cpp
std::vector<Candlestick> yearlyCandlesticks;
double previousClose = 0.0;  // Initialize the previous year's closing value to 0.0

// Iterate through the monthly candlesticks in 12-month chunks
for (size_t i = 0; i < monthlyCandlesticks.size(); i += 12)
{
    Candlestick candle;

    // The "Open" value is the previous year's average temperature
    candle.open = previousClose;

    // The "High" value is the highest temperature recorded in the year
    candle.high = std::max_element(monthlyCandlesticks.begin() + i, monthlyCandlesticks.begin() + std::min(i + 12, monthlyCandlesticks.size()),
                                   [](const Candlestick &a, const Candlestick &b) { return a.high < b.high; })
                  ->high;

    // The "Low" value is the lowest temperature recorded in the year
    candle.low = std::min_element(monthlyCandlesticks.begin() + i, monthlyCandlesticks.begin() + std::min(i + 12, monthlyCandlesticks.size()),
                                  [](const Candlestick &a, const Candlestick &b) { return a.low < b.low; })
                 ->low;

    // The "Close" value is the average temperature of the year
    candle.close = std::accumulate(monthlyCandlesticks.begin() + i, monthlyCandlesticks.begin() + std::min(i + 12, monthlyCandlesticks.size()), 0.0,
                                   [](double sum, const Candlestick &c) { return sum + c.close; }) / 12;

    // Add the candlestick to the yearly candlesticks vector
    yearlyCandlesticks.push_back(candle);

    // Update the previous close value for the next year's open
    previousClose = candle.close;
}
```

Integration of Classes and Objects in Task 1

Class	Implemented In	Functionality
`Candlestick`	`CandlestickCalculator`	Represents the structure of candlestick data.
`CandlestickCalculator`	`WeatherAnalyzer`	Provides methods to calculate candlestick data (daily, monthly, yearly).
`DataParser`	`WeatherAnalyzer`	Parses temperature data from a CSV file and provides input to `CandlestickCalculator`.

Object Usage in `WeatherAnalyzer` Class:

- An instance of `DataParser` is created to load and parse the weather data.

```
// Load the data from the specified CSV file for the chosen country
std::string filePath = "weather_data_EU_1980-2019_temp_only.csv";
DataParser parser(filePath, countryCode);
parser.parse(); // Parse the data
```

- The `CandlestickCalculator` methods (`calculateDailyCandlesticks`, `calculateMonthlyCandlesticks`, `calculateYearlyCandlesticks`) are called within `WeatherAnalyzer` to compute the candlestick data.

```
// Compute candlestick data based on the chosen frequency
std::vector<Candlestick> candlesticks;
if (freq == 'd')
{
    candlesticks = parser.computeDailyCandlesticks();
}
else if (freq == 'm')
{
    candlesticks = parser.computeMonthlyCandlesticks();
}
else if (freq == 'y')
{
    candlesticks = parser.computeYearlyCandlesticks();
}
```

- The computed candlestick data is then displayed and further processed for filtering and plotting.

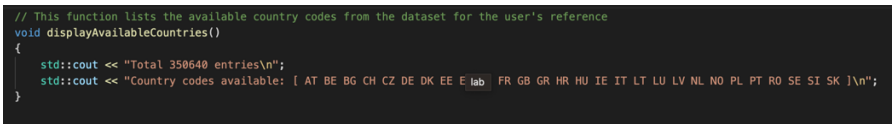
What Happens When Task 1 is Computed

When Task 1 is computed, the process begins with the execution of the `main()` function in the `main.cpp` file.

```cpp
int main() {
    WeatherAnalyzer analyzer;
    analyzer.run();
    return 0;
}
```

The program creates an instance of the `WeatherAnalyzer` class and calls its `run()` method to start the workflow. The `run()` method orchestrates the entire process of computing candlestick data.

The first step in the workflow is displaying the available country codes from the dataset. The `displayAvailableCountries()` method of the `WeatherAnalyzer` class outputs a list of country codes that the user can select from. The user is prompted to enter a country code, which is then used to filter the dataset for that specific country.



Once the user selects a country code, such as `GB` (Great Britain), the program proceeds to load and parse the weather data using the `DataParser` class. The file path to the CSV file containing temperature data is passed to the `DataParser`constructor along with the selected country code. The `parse()` method reads the CSV file and stores the hourly temperature readings in a map, where the keys are dates, and the values are vectors of hourly temperatures.

```cpp
DataParser parser(filePath, countryCode);
parser.parse();
```

After parsing the data, the program computes yearly candlestick data using the `CandlestickCalculator` class. The method `calculateYearlyCandlesticks()` is called on the parsed data to generate candlestick records for each year. This computation involves several steps:

1. Daily candlesticks are first calculated by averaging hourly temperature data over 24-hour chunks.
2. The daily candlesticks are then aggregated into monthly candlesticks by processing 30-day chunks.
3. Finally, yearly candlesticks are computed by aggregating the monthly candlesticks over 12-month periods.

```cpp
std::vector<Candlestick> fullData = parser.computeYearlyCandlesticks();
```

The yearly candlestick data is then displayed in a tabular format. For each year, the table shows the following fields: `Date`, `Open/°C`, `Close/°C`, `High/°C`, and `Low/°C`. The `Open` value represents the average temperature at the start of the year, the `Close` value represents the average temperature at the end of the year, the `High` value is the highest temperature recorded during the year, and the `Low` value is the lowest temperature recorded during the year.

The computed candlestick data is displayed using a loop that iterates through the `fullData` vector. For each `Candlestick`object, the `displayRow()` method is called to format and print the candlestick values. The output includes 40 years of yearly candlestick data for the selected country.

```
for (const auto &candle : fullData) {
    candle.displayRow();
}
```

In summary, the process of computing Task 1 involves selecting a country, parsing the temperature data, calculating daily, monthly, and yearly candlestick values, and displaying the results in a tabular format. Each step is carefully executed using the various classes and methods implemented in the project, ensuring accurate computation and visualization of weather data. Here is a sample output when user chooses GB as country code:

```
Total 350640 entries
Country codes available: [ AT BE BG CH CZ DE DK EE ES FI FR GB GR HR HU IE IT LT LU LV NL NO PL PT RO SE SI SK ]
Enter country code: GB

Displaying data for 40 years:
        Date    Open/°C   Close/°C    High/°C     Low/°C
  1980-01-01          0       3.95    23.054    -3.612
  1981-01-01       3.95      2.404    23.579    -6.533
  1982-01-01      2.404      5.981    23.859    -7.184
  1983-01-01      5.981      7.757     26.22    -3.086
  1984-01-01      7.757       1.63     25.09    -2.993
  1985-01-01       1.63      1.915    23.385    -7.071
  1986-01-01      1.915       4.77    24.636    -5.952
  1987-01-01       4.77      6.213     24.45    -7.127
  1988-01-01      6.213      4.053    23.981    -2.135
  1989-01-01      4.053      1.461     26.49    -1.322
  1990-01-01      1.461      2.236     29.48    -1.765
  1991-01-01      2.236       6.27    24.433    -6.012
  1992-01-01       6.27       0.97    24.382    -2.962
  1993-01-01       0.97      0.424    22.617    -3.535
  1994-01-01      0.424     -0.196    24.679     -3.27
  1995-01-01     -0.196      1.617    27.632    -5.334
  1996-01-01      1.617     -2.856    25.441    -3.433
  1997-01-01     -2.856      3.599    26.192    -3.945
  1998-01-01      3.599      7.069    24.125     -1.96
  1999-01-01      7.069      6.672    25.593    -3.197
  2000-01-01      6.672      2.629    24.307    -1.586
  2001-01-01      2.629     -1.301    24.691    -2.798
  2002-01-01     -1.301      2.776    24.616    -3.338
  2003-01-01      2.776       2.71    28.147    -2.581
  2004-01-01       2.71      5.499    25.029    -2.264
  2005-01-01      5.499      4.105    25.186    -2.238
  2006-01-01      4.105      7.514    28.169     -2.12
  2007-01-01      7.514      6.119    23.289    -2.397
  2008-01-01      6.119     -1.235    24.556     -2.04
  2009-01-01     -1.235     -0.978    25.563    -4.288
  2010-01-01     -0.978      2.623    23.663    -4.609
  2011-01-01      2.623     10.164    23.903    -2.283
  2012-01-01     10.164      5.271    23.794    -5.121
  2013-01-01      5.271      4.726    25.885    -2.771
  2014-01-01      4.726      6.425    24.603    -1.035
  2015-01-01      6.425      3.382    26.586    -2.602
  2016-01-01      3.382      5.753    26.949    -1.399
  2017-01-01      5.753      5.185    25.548     -1.51
  2018-01-01      5.185      7.241    27.342    -4.516
  2019-01-01      7.241      4.343    29.105    -2.719
```

Task 2: Create a Text-Based Plot

Task 2 involves creating a text-based plot to visualize candlestick data (daily, monthly, or yearly). The aim is to provide a simple and scalable way to represent weather trends using text characters. The `CandlestickPlot` class is the primary component responsible for implementing this functionality.

Classes Used in Task 2

1. Candlestick Class

The `Candlestick` class, used in Task 1 to represent candlestick data, is also used in Task 2 to provide the necessary data for plotting. Each `Candlestick` object contains the following fields:

- **`std::string date`**: The date associated with the candlestick.
- **`double open`**: The opening temperature value for the time frame.
- **`double high`**: The highest temperature value for the time frame.
- **`double low`**: The lowest temperature value for the time frame.
- **`double close`**: The closing temperature value for the time frame.

This class provides the data structure necessary for plotting weather trends.

```
// Parameterized constructor to initialize candlestick data with specific values
Candlestick(const std::string &date, double open, double high, double low, double close)
    : date(date), open(open), high(high), low(low), close(close) {}
```

2. CandlestickPlot Class

The `CandlestickPlot` class provides methods to plot candlestick data using text-based charts. It includes functions to generate text-based plots for daily, monthly, or yearly candlestick data and helper functions to print X-axis labels.

Fields and Methods in CandlestickPlot Class

- **`void plot(const std::vector<Candlestick> &candlesticks, int chartHeight, char frequency)`**:
  - Plots the weather candlesticks as a text-based chart.
  - Parameters:
    - **`candlesticks`**: A vector of `Candlestick` objects to be plotted.
    - **`chartHeight`**: The height of the text chart in lines.
    - **`frequency`**: The time period frequency ('d' for daily, 'm' for monthly, 'y' for yearly).

Code Snippet for `plot()` Method

```cpp
void CandlestickPlot::plot(const std::vector<Candlestick> &candlesticks, int chartHeight, char frequency)
{
    // Check if the input candlestick data is empty
    if (candlesticks.empty())
    {
        std::cerr << "No data to plot." << std::endl;
        return;
    }

    // Initialize variables to track the minimum and maximum temperature values for the Y-axis
    double minTemp = candlesticks.front().low;
    double maxTemp = candlesticks.front().high;

    // Loop through all candlesticks to find the overall minimum and maximum temperatures
    for (const auto &candle : candlesticks)
    {
        if (candle.low < minTemp)
            minTemp = candle.low;
        if (candle.high > maxTemp)
            maxTemp = candle.high;
    }

    // Calculate the range of temperatures and the step size for each row in the chart
    double range = maxTemp - minTemp;
    double step = range / chartHeight;
```

```cpp
        else if (std::abs(currentTemp - candle.close) < step / 2.0)
        {
            std::cout << "----"; // Close value
        }
        else if (currentTemp < candle.high && currentTemp > candle.low)
        {
            std::cout << " | "; // Inside stalk
        }
        else
        {
            std::cout << "     "; // Empty space
        }
    }

    // Move to the next row
    std::cout << std::endl;
}
```

```cpp
// Print the header of the chart
std::cout << "\n====================== Weather Candlestick Chart ======================\n";

// Loop to generate the plot row by row
for (int i = chartHeight; i >= 0; --i)
{
    // Calculate the current temperature for this row
    double currentTemp = minTemp + i * step;

    // Print the Y-axis label with temperature in °C
    std::cout << std::setw(6) << std::fixed << std::setprecision(1) << currentTemp << "°C | ";

    // Loop through each candlestick to plot the corresponding symbols for high, low, open, and close values
    for (size_t j = 0; j < candlesticks.size(); ++j)
    {
        const auto &candle = candlesticks[j];

        // Add spacing between candlesticks for better readability
        std::cout << std::setw(5);

        // Determine which symbol to plot based on the current temperature and candlestick values
        if (std::abs(currentTemp - candle.high) < step / 2.0)
        {
            std::cout << " | "; // High stalk
        }
        else if (std::abs(currentTemp - candle.low) < step / 2.0)
        {
            std::cout << " | "; // Low stalk
        }
        else if (std::abs(currentTemp - candle.open) < step / 2.0)
        {
            std::cout << "----"; // Open value
        }
```

3. WeatherAnalyzer Class

The `WeatherAnalyzer` class integrates the plotting functionality. It contains methods to compute and plot candlestick data without requiring the user to choose a frequency in Task 1.

- **`void plotMultipleGraphs(const std::vector<Candlestick> &candlesticks, int chartHeight, char frequency)`**:
  - Plots multiple graphs by dividing the candlestick data into chunks.
  - Automatically generates a graph when Task 1 is executed without additional user input.

Integration of Classes and Objects in Task 2

Class	Implemented In	Functionality
`Candlestick`	`CandlestickPlot`	Provides the data structure for plotting candlestick data.
`CandlestickPlot`	`WeatherAnalyzer`	Generates text-based plots for candlestick data.
`WeatherAnalyzer`	`main.cpp`	Integrates the plotting functionality and prompts user input.

What Happens When Task 2 is Computed

When Task 2 is computed, the process begins with the execution of the `run()` method in the `WeatherAnalyzer` class. This method orchestrates the workflow by prompting the user to select the frequency of the candlestick data to be plotted. The available options include daily, monthly, and yearly candlestick data. Once the frequency is selected, the `plotMultipleGraphs()` function is called to visualize the data.

However, it is important to note that even without selecting a frequency, Task 1 automatically generates a default text-based plot to display the computed candlestick data. This ensures that users are provided with a visual representation of the weather trends without any additional input.

Step 1: User Input for Frequency

The user is prompted to choose the frequency of the candlestick data they wish to plot. The options are:

- `d`: Daily candlestick data
- `m`: Monthly candlestick data
- `y`: Yearly candlestick data

The selected frequency determines which candlestick data is passed to the `CandlestickPlot::plot()` method. The following code snippet shows how the program captures the user's input for frequency:

```
char freq = promptFrequency();
std::vector<Candlestick> candlesticks;
if (freq == 'd') {
    candlesticks = parser.computeDailyCandlesticks();
} else if (freq == 'm') {
    candlesticks = parser.computeMonthlyCandlesticks();
} else if (freq == 'y') {
    candlesticks = parser.computeYearlyCandlesticks();
} else {
    std::cerr << "Invalid frequency entered. Exiting." << std::endl;
    return;
}
```

Step 2: Data Processing and Plotting

Once the frequency is selected, the program retrieves the corresponding candlestick data from the `DataParser` class. The `CandlestickPlot::plot()` method is then called to generate a text-based plot. This method dynamically adjusts the Y-axis based on the temperature ranges in the data and uses specific symbols to represent the open, high, low, and close values for each candlestick.

```
CandlestickPlot::plot(candlesticks, 20, freq);
```

The chart dynamically scales according to the temperature range, making it easier to interpret the trends.

Step 3: Generating the Chart

The `plot()` method in the `CandlestickPlot` class uses the following symbols to represent the candlestick data:

- `'|'`: Represents the high and low stalks.
- `'----'`: Represents the open and close values.

The method iterates through the temperature values and prints the chart row by row. Each row corresponds to a specific temperature value on the Y-axis, and the corresponding symbols are displayed for each candlestick.

```
// Loop to generate the plot row by row
for (int i = chartHeight; i >= 0; --i)
{
    // Calculate the current temperature for this row
    double currentTemp = minTemp + i * step;

    // Print the Y-axis label with temperature in °C
    std::cout << std::setw(6) << std::fixed << std::setprecision(1) << currentTemp << "°C | ";

    // Loop through each candlestick to plot the corresponding symbols for high, low, open, and close values
    for (size_t j = 0; j < candlesticks.size(); ++j)
    {
        const auto &candle = candlesticks[j];

        // Add spacing between candlesticks for better readability
        std::cout << std::setw(5);

        // Determine which symbol to plot based on the current temperature and candlestick values
        if (std::abs(currentTemp - candle.high) < step / 2.0)
        {
            std::cout << " | "; // High stalk
        }
        else if (std::abs(currentTemp - candle.low) < step / 2.0)
        {
            std::cout << " | "; // Low stalk
        }
        else if (std::abs(currentTemp - candle.open) < step / 2.0)
        {
            std::cout << "----"; // Open value
        }
        else if (std::abs(currentTemp - candle.close) < step / 2.0)
        {
```

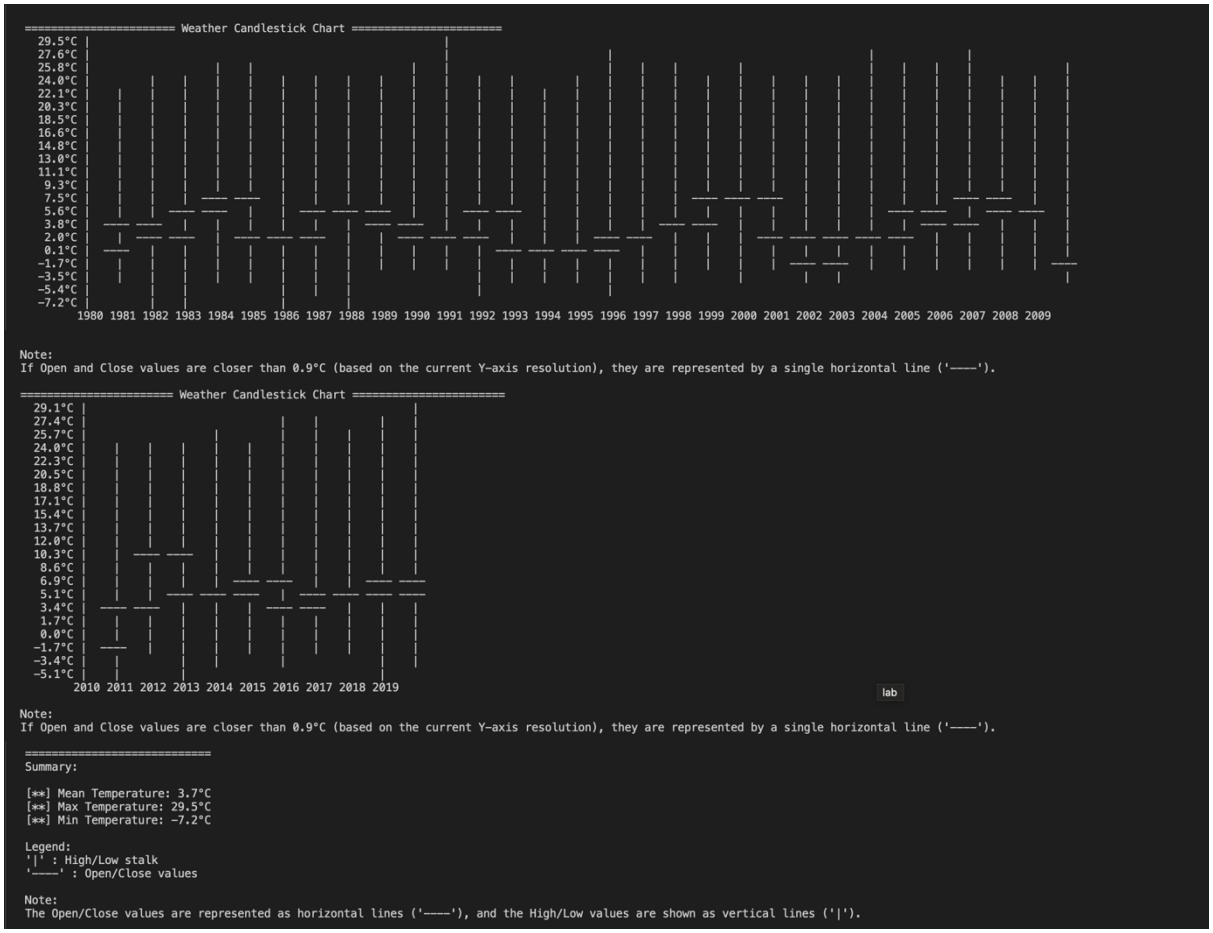
Step 4: Displaying the Chart

Once the chart is generated, it is displayed in the console. This allows users to visualize weather trends over the selected time frame in a text-based format. The automatic graph generation in Task 1 ensures that users receive a visual output even without selecting a frequency.

```
// Print the header of the chart
std::cout << "\n====================== Weather Candlestick Chart ======================\n";

// Loop to generate the plot row by row
for (int i = chartHeight; i >= 0; --i)
```

In summary, Task 2 generates a text-based candlestick plot representing weather trends over time. The user is prompted to select the frequency (daily, monthly, or yearly) of the candlestick data, which is then passed to the `CandlestickPlot` class to dynamically adjust the Y-axis based on temperature ranges. Symbols such as '|' and '----' are used to represent the high/low and open/close values, respectively. Additionally, the chart is automatically generated after Task 1 to ensure users receive a visual output even without selecting a frequency. Here is the output generated for country code GB Task 1 data representation:



Task 3: Filter Data

Task 3 involves filtering the candlestick data based on various criteria, such as date range, temperature range, and specific months. The `Filters` class is the primary component responsible for implementing the filtering functionality.

Classes Used in Task 3

1. Filters Class

The `Filters` class provides static methods to filter candlestick data based on different conditions. It includes functions for filtering by date range, month, minimum temperature, maximum temperature, and temperature range.

Fields and Methods in Filters Class

- **`filterByDateRange(const std::vector<Candlestick> &candlesticks, const std::string &startDate, const std::string &endDate)`**:
    - Filters the candlestick data within a specified date range.

- **`filterByMonthAcrossYears(const std::vector<Candlestick> &candlesticks, const std::string &targetMonth)`**:
  - Filters candlestick data by a specific month across multiple years.
- **`filterByMinTemperature(const std::vector<Candlestick> &candlesticks, double minTemp)`**:
  - Filters candlestick data where the temperature is above a specified minimum value.
- **`filterByMaxTemperature(const std::vector<Candlestick> &candlesticks, double maxTemp)`**:
  - Filters candlestick data where the temperature is below a specified maximum value.
- **`filterByTemperatureRange(const std::vector<Candlestick> &candlesticks, double minTemp, double maxTemp)`**:
  - Filters candlestick data within a specified temperature range.

Code Snippet for Filters:

```cpp
std::vector<Candlestick> Filters::filterByDateRange(const std::vector<Candlestick> &candlesticks, const std::string &startDate, const std::string &endDate)
{
    std::vector<Candlestick> filtered; // Vector to store filtered data

    // Iterate over each candlestick to check if its date falls within the specified range
    for (const auto &candle : candlesticks)
    {
        if (candle.date >= startDate && candle.date < endDate)
        {
            filtered.push_back(candle); // Add the candlestick to the filtered list if it matches the criteria
        }
    }
    return filtered; // Return the filtered vector
}
std::vector<Candlestick> Filters::filterByMonthAcrossYears(const std::vector<Candlestick> &candlesticks, const std::string &targetMonth)
{
    std::vector<Candlestick> filtered; // Vector to store filtered data

    // Iterate over each candlestick to check if its date contains the target month
    for (const auto &candle : candlesticks)
    {
        if (candle.date.substr(5, 2) == targetMonth)
        {
            filtered.push_back(candle); // Add the candlestick to the filtered list if it matches the criteria
        }
    }
    return filtered; // Return the filtered vector
}
```

2. WeatherAnalyzer Class

The `WeatherAnalyzer` class integrates the filtering functionality. It includes methods to prompt the user for filter criteria and apply the selected filters to the candlestick data.

- **`void applyAdditionalFilters(std::vector<Candlestick> &candlesticks, char frequency)`**:
  - Allows users to apply various filters to the candlestick data.
  - Prompts the user to choose a filter type and applies the corresponding filter.

Code Snippet for Applying Filters

```cpp
int choice; // Variable to store user's choice for filter options
do
{
    // Display filter options to the user
    std::cout << "\nSelect a filter option:\n";
    std::cout << "1. Filter by Month Across All Years\n";
    std::cout << "2. Filter by Minimum Temperature\n";
    std::cout << "3. Filter by Maximum Temperature\n";
    std::cout << "4. Filter by Temperature Range\n";
    std::cout << "5. No Filter (Display All Data)\n";
    std::cout << "Enter your choice: ";
    std::cin >> choice; // Get the user's choice

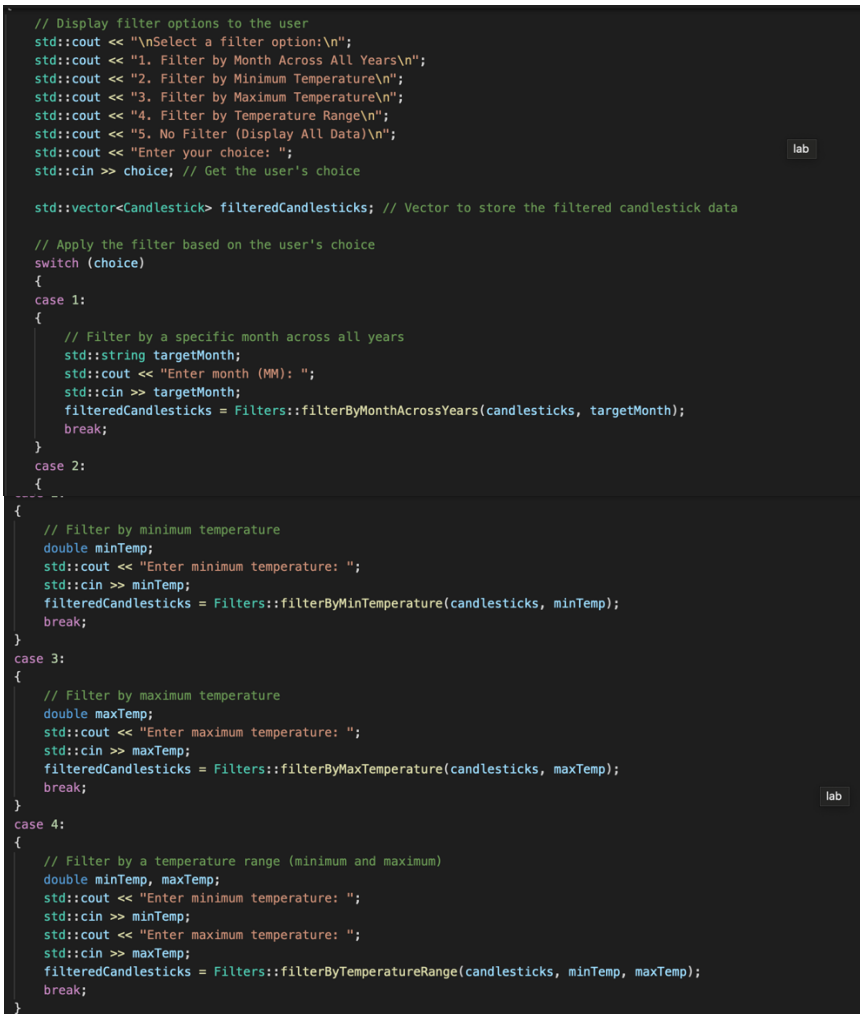
    std::vector<Candlestick> filteredCandlesticks; // Vector to store the filtered candlestick data

    // Apply the filter based on the user's choice
    switch (choice)
```

What Happens When Task 3 is Computed

When Task 3 is computed, the process begins by prompting the user with a list of available filtering options. The options include filtering by month, minimum temperature, maximum temperature, or a specific temperature range. The `applyAdditionalFilters()` method in the `WeatherAnalyzer` class orchestrates this process by displaying the filter choices and capturing user input. Based on the user's choice, a corresponding filter function from the `Filters` class is called to apply the selected filter to the candlestick data.

For instance, if the user chooses to filter by month, the program prompts the user to enter a specific month in MM format. The `filterByMonthAcrossYears()` method from the `Filters` class is then invoked, iterating through the candlestick data and extracting only those records that match the specified month. Similarly, for filtering by minimum or maximum temperature,
the `filterByMinTemperature()` and `filterByMaxTemperature()` methods are used to return only the candlesticks that meet the temperature criteria. Here's a code snippet demonstrating the application of filters:



```
// Display filter options to the user
std::cout << "\nSelect a filter option:\n";
std::cout << "1. Filter by Month Across All Years\n";
std::cout << "2. Filter by Minimum Temperature\n";
std::cout << "3. Filter by Maximum Temperature\n";
std::cout << "4. Filter by Temperature Range\n";
std::cout << "5. No Filter (Display All Data)\n";
std::cout << "Enter your choice: ";
std::cin >> choice; // Get the user's choice

std::vector<Candlestick> filteredCandlesticks; // Vector to store the filtered candlestick data

// Apply the filter based on the user's choice
switch (choice)
{
case 1:
{
    // Filter by a specific month across all years
    std::string targetMonth;
    std::cout << "Enter month (MM): ";
    std::cin >> targetMonth;
    filteredCandlesticks = Filters::filterByMonthAcrossYears(candlesticks, targetMonth);
    break;
}
case 2:
{
    // Filter by minimum temperature
    double minTemp;
    std::cout << "Enter minimum temperature: ";
    std::cin >> minTemp;
    filteredCandlesticks = Filters::filterByMinTemperature(candlesticks, minTemp);
    break;
}
case 3:
{
    // Filter by maximum temperature
    double maxTemp;
    std::cout << "Enter maximum temperature: ";
    std::cin >> maxTemp;
    filteredCandlesticks = Filters::filterByMaxTemperature(candlesticks, maxTemp);
    break;
}
case 4:
{
    // Filter by a temperature range (minimum and maximum)
    double minTemp, maxTemp;
    std::cout << "Enter minimum temperature: ";
    std::cin >> minTemp;
    std::cout << "Enter maximum temperature: ";
    std::cin >> maxTemp;
    filteredCandlesticks = Filters::filterByTemperatureRange(candlesticks, minTemp, maxTemp);
    break;
}
```

Once the filter is applied, the filtered candlestick data is displayed in a tabular format using the `Candlestick::displayRow()` method. The filtered data is also plotted on a text-based graph using the `CandlestickPlot::plot()` method, providing the user with a visual representation of the filtered results. The process continues, allowing the user to apply additional filters or stop the filtering process at any time by selecting the "No Filter" option.

Concludingly, in Task 3, the program allows users to apply various filters to the candlestick data, such as filtering by month, minimum temperature, or temperature range. The `applyAdditionalFilters()` method in the `WeatherAnalyzer` class handles the user interaction by displaying filter options and capturing their choice. Based on the selection, the corresponding filter method from the `Filters` class is applied to the data, such as `filterByDateRange()` or `filterByTemperatureRange()`. The filtered data is then displayed in a table format using the `Candlestick::displayRow()` method and visually represented as a text-based plot using the `CandlestickPlot::plot()` method. This process continues until the user chooses to stop applying filters. Here is an example output:

```
Would you like to start filtering? (y/n): y
 Enter start date (inclusive, e.g., 2018, 1990-02, 2019-12-22): 2000
Enter end date (exclusive, e.g., 2018, 1990-02, 2019-12-22): 2004
Enter frequency (y(yearly), m(monthly), d(daily)): m
 How many entries do you want to display? (Enter 0 to display all): 0

Would you like to apply more filters? (y/n): y

Select a filter option:
1. Filter by Month Across All Years
2. Filter by Minimum Temperature
3. Filter by Maximum Temperature
4. Filter by Temperature Range
5. No Filter (Display All Data)
Enter your choice: 2
Enter minimum temperature: -2
```

Open, High, Low, and Close values for the filtered range:

Date	Open/°C	Close/°C	High/°C	Low/°C
2000-01-01	6.7	7.8	10.5	0.0
2000-02-01	7.8	3.6	10.3	-0.6
2000-03-01	3.6	3.3	13.9	-1.5
2000-04-01	3.3	7.6	14.9	-0.6
2000-05-01	7.6	10.6	21.1	5.1
2000-06-01	10.6	12.2	24.3	7.5
2000-07-01	12.2	15.6	22.3	7.8
2000-08-01	15.6	13.5	22.8	9.6
2000-09-01	13.5	10.9	23.1	7.3
2000-10-01	10.9	7.0	17.3	5.3
2000-11-01	7.0	9.5	12.5	2.7
2000-12-01	9.5	2.6	11.6	-1.6
2001-01-01	2.6	0.5	8.5	-2.0
2001-02-01	0.5	-1.1	10.9	-1.9
2001-04-01	7.6	4.6	14.5	0.4
2001-05-01	4.6	9.2	20.6	3.0
2001-06-01	9.2	13.1	23.0	4.6
2001-07-01	13.1	13.5	24.7	8.4
2001-08-01	13.5	11.7	24.0	9.2
2001-09-01	11.7	15.3	19.7	7.3
2001-10-01	15.3	6.1	19.0	6.1
2001-11-01	6.1	11.8	13.4	0.3
2002-02-01	6.3	1.9	11.5	0.0
2002-03-01	1.9	8.5	13.4	0.0
2002-04-01	8.5	6.0	17.4	1.3
2002-05-01	6.0	10.7	20.7	3.2
2002-06-01	10.7	11.6	21.1	7.3
2002-07-01	11.6	14.9	24.6	8.6
2002-08-01	14.9	10.9	24.6	10.8
2002-09-01	10.9	11.5	20.8	7.4
2002-10-01	11.5	10.2	19.1	2.4
2002-11-01	10.2	8.4	13.8	4.7
2002-12-01	8.4	2.8	11.2	-0.3
2003-03-01	6.3	5.0	14.1	-0.0
2003-04-01	5.0	7.5	20.7	-1.4
2003-05-01	7.5	13.4	22.6	3.3
2003-06-01	13.4	13.0	21.9	8.4
2003-07-01	13.0	15.0	25.5	10.4
2003-08-01	15.0	10.1	28.1	7.4
2003-09-01	10.1	11.2	22.6	5.3

Task 4: Predict Future Temperatures

Task 4 involves predicting future temperatures based on historical weather data using the **moving average method**. The prediction is displayed as candlestick data, representing future temperature trends. This task uses several classes and methods from the project, including the `Predictor`, `WeatherAnalyzer`, and `Candlestick` classes. Below is a detailed explanation of each class, constructor, object, and function involved in Task 4, followed by an in-depth discussion of the moving average method, its implementation in the code, and a comparison with other prediction methods.

Classes Used in Task 4

1. Candlestick Class

The `Candlestick` class represents a single candlestick object containing date, open, high, low, and close values. It provides the structure required to store the predicted temperature data.

Fields:

- **`std::string date`**: The date associated with the candlestick.
- **`double open`**: The predicted opening temperature for the time frame.
- **`double high`**: The predicted highest temperature for the time frame.
- **`double low`**: The predicted lowest temperature for the time frame.
- **`double close`**: The predicted closing temperature for the time frame.

Constructor:

The parameterized constructor initializes the candlestick data with specific values for date, open, high, low, and close temperatures.

```
// Parameterized constructor to initialize candlestick data with specific values
Candlestick(const std::string &date, double open, double high, double low, double close)
    : date(date), open(open), high(high), low(low), close(close) {}
```

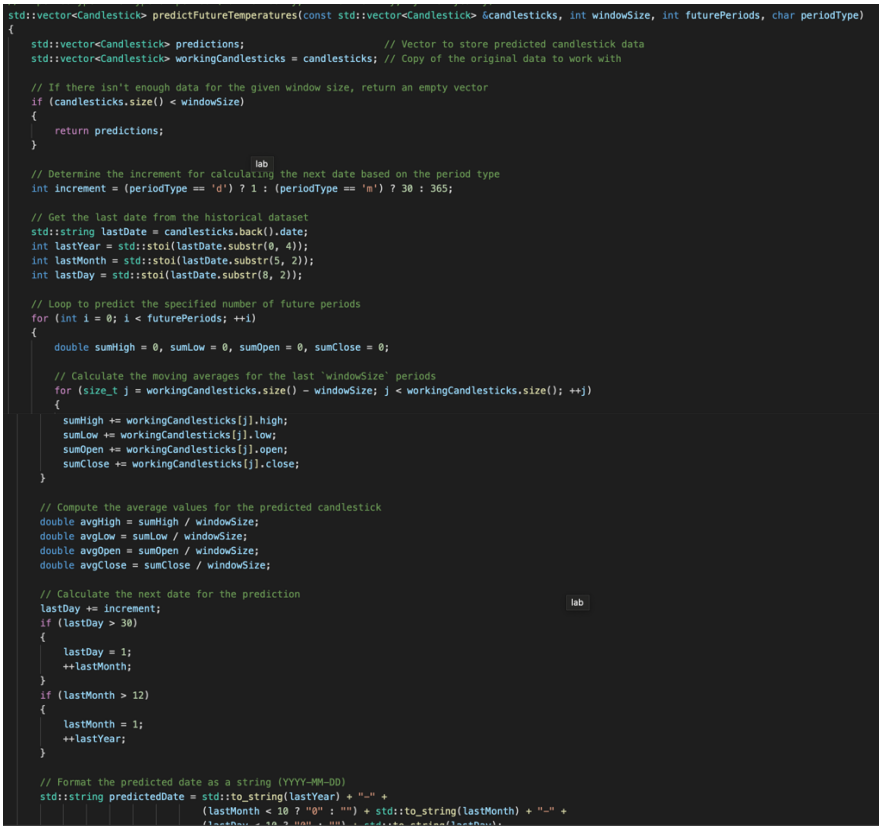
2. Predictor Class

The `Predictor` class provides the function to predict future temperatures using the moving average method. It takes historical candlestick data and generates predictions for a specified number of future periods.

Function:

- **`std::vector<Candlestick> predictFutureTemperatures()`**: Predicts future temperatures based on historical candlestick data using a moving average approach.

Code Snippet:



```cpp
std::vector<Candlestick> predictFutureTemperatures(const std::vector<Candlestick> &candlesticks, int windowSize, int futurePeriods, char periodType)
{
    std::vector<Candlestick> predictions;                    // Vector to store predicted candlestick data
    std::vector<Candlestick> workingCandlesticks = candlesticks; // Copy of the original data to work with

    // If there isn't enough data for the given window size, return an empty vector
    if (candlesticks.size() < windowSize)
    {
        return predictions;
    }

    // Determine the increment for calculating the next date based on the period type
    int increment = (periodType == 'd') ? 1 : (periodType == 'm') ? 30 : 365;

    // Get the last date from the historical dataset
    std::string lastDate = candlesticks.back().date;
    int lastYear = std::stoi(lastDate.substr(0, 4));
    int lastMonth = std::stoi(lastDate.substr(5, 2));
    int lastDay = std::stoi(lastDate.substr(8, 2));

    // Loop to predict the specified number of future periods
    for (int i = 0; i < futurePeriods; ++i)
    {
        double sumHigh = 0, sumLow = 0, sumOpen = 0, sumClose = 0;

        // Calculate the moving averages for the last `windowSize` periods
        for (size_t j = workingCandlesticks.size() - windowSize; j < workingCandlesticks.size(); ++j)
        {
            sumHigh += workingCandlesticks[j].high;
            sumLow += workingCandlesticks[j].low;
            sumOpen += workingCandlesticks[j].open;
            sumClose += workingCandlesticks[j].close;
        }

        // Compute the average values for the predicted candlestick
        double avgHigh = sumHigh / windowSize;
        double avgLow = sumLow / windowSize;
        double avgOpen = sumOpen / windowSize;
        double avgClose = sumClose / windowSize;

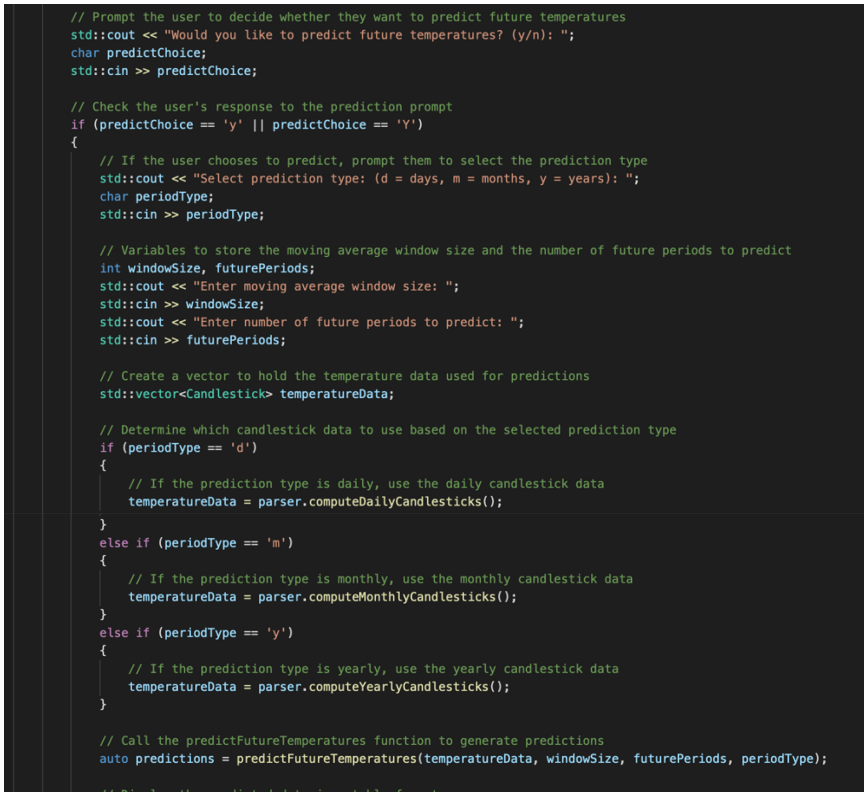
        // Calculate the next date for the prediction
        lastDay += increment;
        if (lastDay > 30)
        {
            lastDay = 1;
            ++lastMonth;
        }
        if (lastMonth > 12)
        {
            lastMonth = 1;
            ++lastYear;
        }

        // Format the predicted date as a string (YYYY-MM-DD)
        std::string predictedDate = std::to_string(lastYear) + "-" +
                                    (lastMonth < 10 ? "0" : "") + std::to_string(lastMonth) + "-" +
```

3. WeatherAnalyzer Class

The `WeatherAnalyzer` class integrates the prediction functionality by calling the `predictFutureTemperatures()` function.

Function Call:



```
// Prompt the user to decide whether they want to predict future temperatures
std::cout << "Would you like to predict future temperatures? (y/n): ";
char predictChoice;
std::cin >> predictChoice;

// Check the user's response to the prediction prompt
if (predictChoice == 'y' || predictChoice == 'Y')
{
    // If the user chooses to predict, prompt them to select the prediction type
    std::cout << "Select prediction type: (d = days, m = months, y = years): ";
    char periodType;
    std::cin >> periodType;

    // Variables to store the moving average window size and the number of future periods to predict
    int windowSize, futurePeriods;
    std::cout << "Enter moving average window size: ";
    std::cin >> windowSize;
    std::cout << "Enter number of future periods to predict: ";
    std::cin >> futurePeriods;

    // Create a vector to hold the temperature data used for predictions
    std::vector<Candlestick> temperatureData;

    // Determine which candlestick data to use based on the selected prediction type
    if (periodType == 'd')
    {
        // If the prediction type is daily, use the daily candlestick data
        temperatureData = parser.computeDailyCandlesticks();
    }
    else if (periodType == 'm')
    {
        // If the prediction type is monthly, use the monthly candlestick data
        temperatureData = parser.computeMonthlyCandlesticks();
    }
    else if (periodType == 'y')
    {
        // If the prediction type is yearly, use the yearly candlestick data
        temperatureData = parser.computeYearlyCandlesticks();
    }

    // Call the predictFutureTemperatures function to generate predictions
    auto predictions = predictFutureTemperatures(temperatureData, windowSize, futurePeriods, periodType);
```

Why Moving Average Was Selected

The moving average method was chosen for its simplicity and effectiveness in forecasting time-series data. It smoothens fluctuations in historical data by averaging a specific number of previous data points, making it suitable for identifying trends.

Advantages of Moving Average:

1. **Easy to Implement**: The method is simple and requires minimal computation.
2. **Smoothens Data**: It reduces the impact of short-term fluctuations in temperature data, making trends more visible.
3. **Effective for Seasonal Data**: The method works well with weather data, which often exhibits seasonal patterns.

Why Other Methods Were Not Preferred:

1. **Exponential Smoothing**: While more advanced, it requires more complex calculations and parameter tuning.
2. **ARIMA Models**: These are powerful for time-series forecasting but are computationally intensive and require careful model selection.
3. **Machine Learning Models**: Though highly accurate, these methods would require more data and computational resources.

How Moving Average is Used in the Code

In the `predictFutureTemperatures()` function, the moving average is calculated by summing up the `high`, `low`, `open`, and `close` values from the last `windowSize` periods and dividing by the window size to get the average. This average is then used to predict future candlestick values.

Summary

In Task 4, future temperature trends are predicted using a moving average method. The `Predictor` class implements this functionality, and the `WeatherAnalyzer` class integrates it into the program workflow. The moving average method was chosen for its simplicity and effectiveness in smoothing out fluctuations in weather data. While other forecasting methods exist, moving average is computationally efficient and well-suited for identifying long-term trends in seasonal data like temperature. Here is an example of output:

```
Would you like to apply more filters? (y/n): n
Would you like to predict future temperatures? (y/n): y
Select prediction type: (d = days, m = months, y = years): m
Enter moving average window size: 10
Enter number of future periods to predict: 30


Predicted Data:
        Date    Open/°C   Close/°C    High/°C     Low/°C
  2020-01-01       9.1        8.9       19.5        4.1
  2020-02-01       9.4        9.5       20.1        4.3
  2020-03-01      10.0        9.6       20.1        4.8
  2020-04-01      10.2        9.4       20.2        5.1
  2020-05-01      10.1        9.1       19.7        4.8
  2020-06-01       9.8        8.5       18.7        4.3
  2020-07-01       9.3        8.3       17.9        3.8
  2020-08-01       9.1        7.7       17.6        3.5
  2020-09-01       8.6        7.7       17.8        3.6
  2020-10-01       8.7        8.3       18.3        3.9
  2020-11-01       9.4        8.7       19.0        4.2
  2020-12-01       9.5        8.7       18.9        4.2
  2021-01-01       9.5        8.6       18.8        4.2
  2021-02-01       9.4        8.5       18.7        4.2
  2021-03-01       9.3        8.4       18.5        4.1
  2021-04-01       9.3        8.3       18.4        4.0
  2021-05-01       9.2        8.3       18.4        4.0
  2021-06-01       9.2        8.3       18.4        4.0
  2021-07-01       9.2        8.4       18.5        4.0
  2021-08-01       9.3        8.5       18.6        4.1
  2021-09-01       9.3        8.5       18.6        4.1
  2021-10-01       9.3        8.5       18.6        4.1
  2021-11-01       9.3        8.4       18.6        4.1
  2021-12-01       9.3        8.4       18.5        4.1
  2022-01-01       9.3        8.4       18.5        4.0
  2022-02-01       9.3        8.4       18.5        4.0
  2022-03-01       9.3        8.4       18.5        4.1
  2022-04-01       9.3        8.4       18.6        4.1
  2022-05-01       9.3        8.4       18.6        4.1
  2022-06-01       9.3        8.4       18.6        4.1
```